



Loyola Marymount
University

CMSI 5350/EECE 5150 Machine Learning

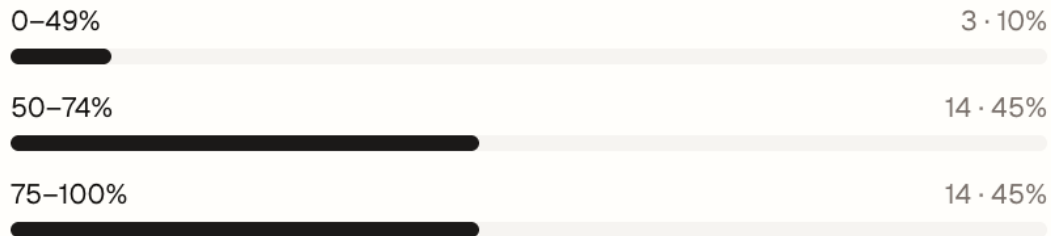
Professor Lanyu Shang

Announcements

- PostCommit Quiz
 - Scores are available on PostCommit and Brightspace
 - Feedback and PCQ COACH
 - Lowest score will be dropped

Class score distribution

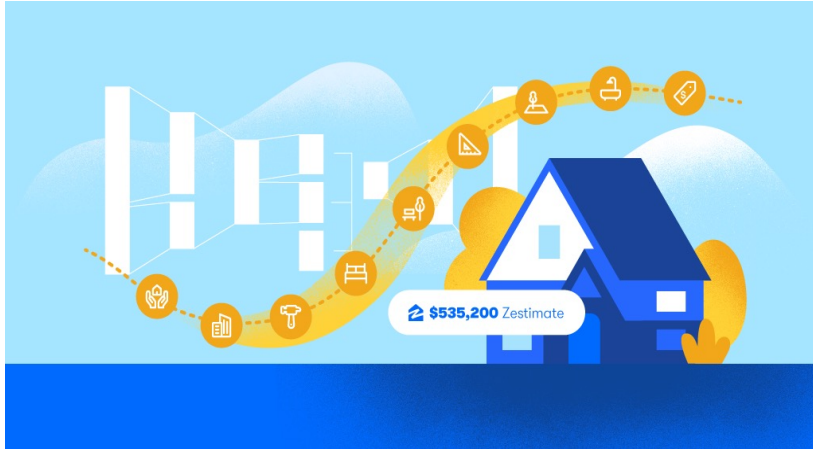
Attempt percentages include every question and use instructor-final grades.



This lecture

- K-nearest neighbor (KNN)
 - Algorithm
 - Impact of k
 - KNN variants
- Implementation
 - Decision Tree
 - KNN

You are your neighbors



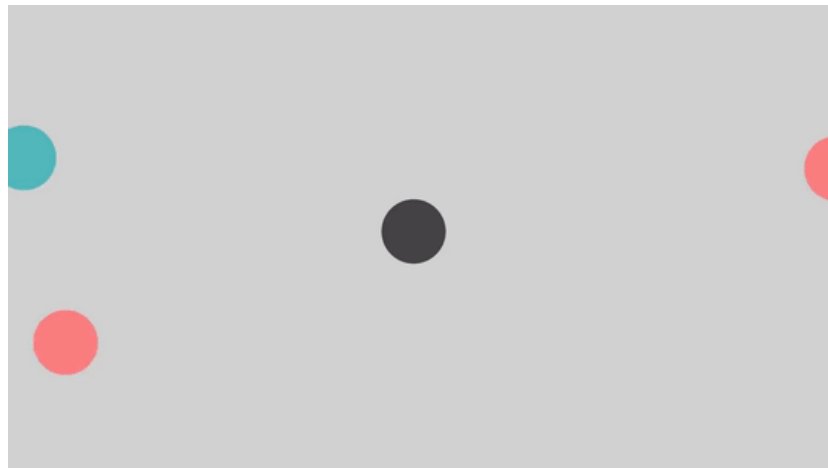
Physical Neighbors



Behavioral Neighbors

K-Nearest Neighbors (KNN)

- How to predict example x :
 - Find **k** nearest neighbors of x
 - Predict
 - **Classification**: Vote by **majority** class
 - **Regression**: **Average** of neighbor values



Distance metrics

- Euclidean distance

$$d(x,y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

Works well for continuous features

- Manhattan distance
 - Taxi distance

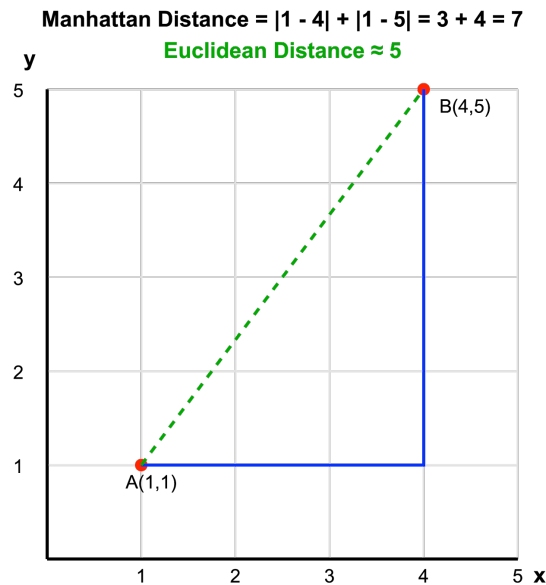
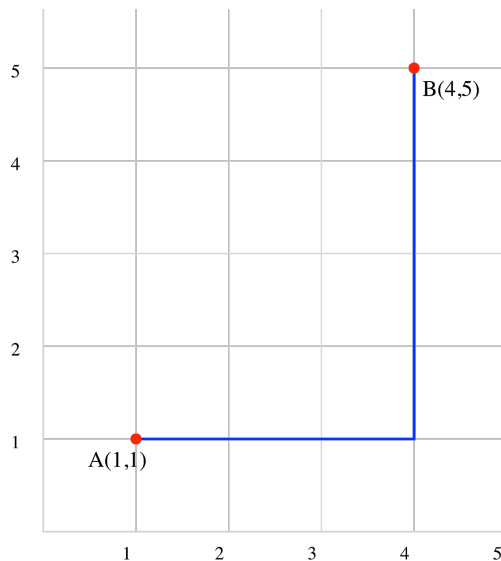
$$d(x,y) = \left(\sum_{i=1}^m |x_i - y_i| \right)$$

Works well for:

- High dimensional features
- Sparse features

In 2D

- What is the Euclidean and Manhattan distance between A and B?



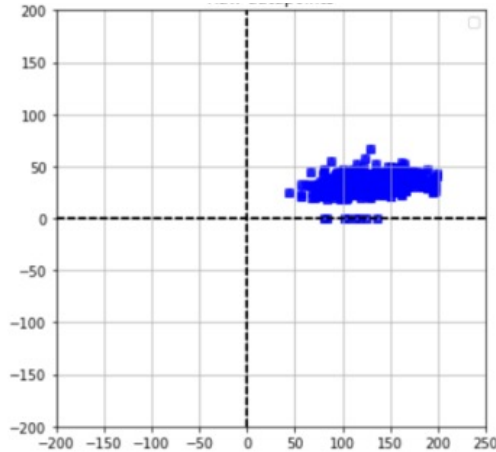
Feature normalization

$$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

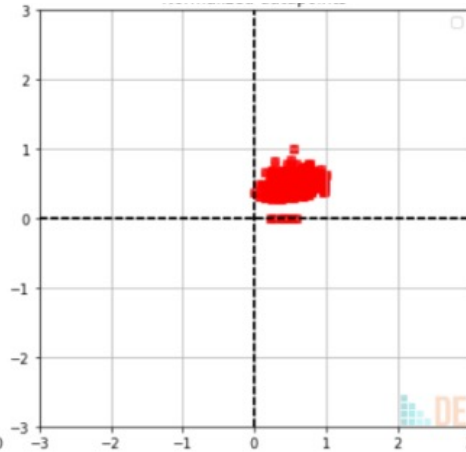
Min-max

$$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation}(x)}$$

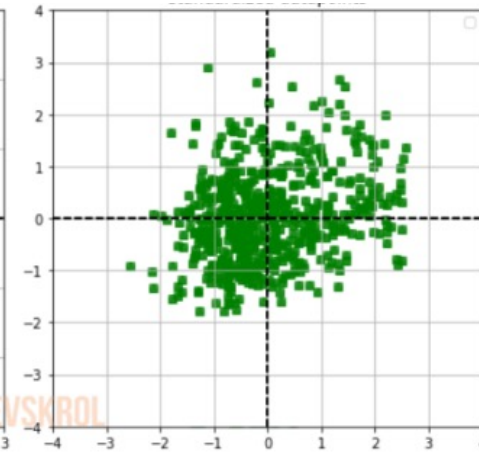
Standard Normalization



Raw



Min-max



Standard

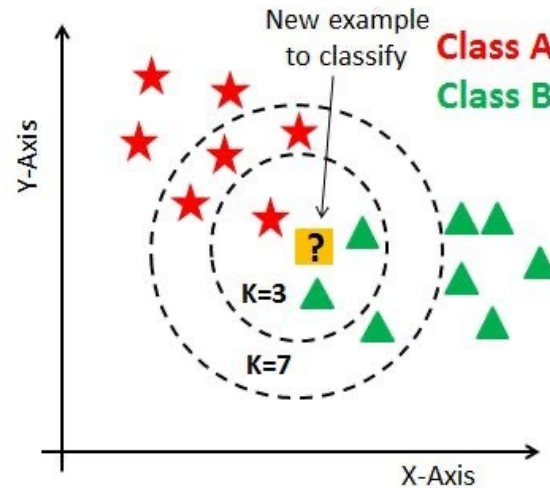
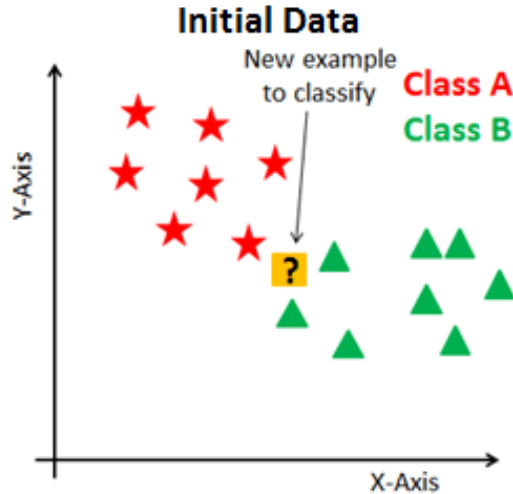
Raw?
Min-max?
Standard?

Preserves relationships
Sensitive to **outliers**

Assumes **normal** distribution
Less sensitive to outliers

The impact of k in KNN

- How to classify the new data point?



The great k debate: small vs. big k

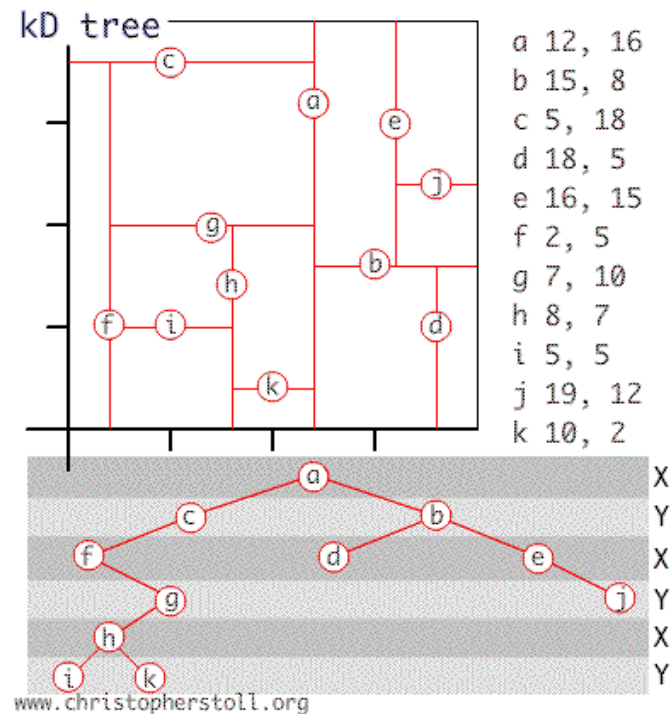
- $K=1$?
 - Risk of overfitting
- $K=\text{large } N$?
 - Underfitting – always the majority class
- How to determine ?
 - Validation set!
 - Common heuristic : choose 3,5,7 (odd number, why?)

Decision tree vs. KNN

- Which is faster to train?
 - k-NN does not require any training!
 - K-NN is a “lazy learning” or “instance-based” algorithm
- Which is faster to predict?
 - For most data sets, decision trees.
 - For each new query point, KNN finds the k nearest neighbors in the stored training data.

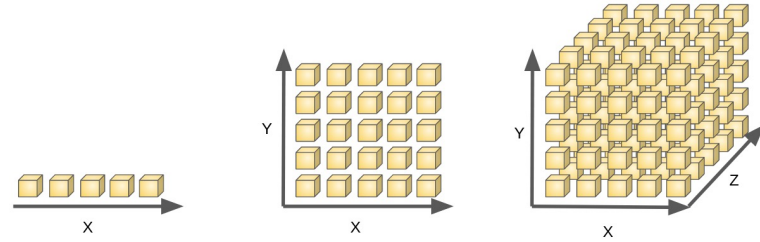
Decision tree vs. KNN

- Do they use features in same way to label examples?
 - k-NN treats all features equally!
 - Decision trees “select” important features.
- Any ideas to improve the prediction efficiency of KNN?
 - KD tree (k-dimensional tree)



The curse of dimensionality

- What is a dimension?
 - In ML, each feature is a dimension.
- The problem
 - In high-dimensional spaces, the distance between any two points becomes increasingly similar - the **nearest** and **farthest** neighbors end up being roughly the same distance apart.
 - The volume of the space grows **exponentially** with each added dimension, but the number of data points typically doesn't grow at the same rate.



The curse of dimensionality

- Practical consequences for KNN
 - Poor performance
 - Increased computational cost
 - Required training data grows exponentially
- Solutions?
 - Dimensionality reduction (PCA, t-SNE)
 - Feature engineering -> meaningful features
 - Increase training data

Summary

- Pros
 - Easy to **implement**
 - **Adapts** easily when training samples are added
 - Few **hyperparameters** – only the value of k
- Cons
 - Does **not scale** well – a lazy algorithm
 - Curse of **dimensionality** -- not work well w/ high-dimensional data
 - Prone to **overfitting**
 - **Low interpretability**

Colab Notebook

DECISION TREE & KNN